Realization of Long Short-Term Memory Networks on Quantum Circuits

Xiaokai Hou
Institute of Fundamental and Frontier Sciences
University of Electronic Science and Technology of China
Chengdu, China
xiaokaihou615@std.uestc.edu.cn

Yingli Yang
Institute of Fundamental and Frontier Sciences
University of Electronic Science and Technology of China
Chengdu, China
yangyingli1999@gmail.com

Xiaoting Wang
Institute of Fundamental and Frontier Sciences
University of Electronic Science and Technology of China
Chengdu, China
xiaoting@uestc.edu.cn

***Abstract*—As one of the well-known methods to solve natural language processing (NLP) problems, the long short-term memory neural network (LSTM) has been designed to implement on quantum computers. However, existing method requires the number of qubits is equal to the dimension of the word vectors, and may result in a surge of the qubit number. To address this problem, we propose a duplication-free quantum long short-term memory neural network (DQLSTM). Specifically, our DQLSTM adopts the amplitude encoding method to store the classical information, and further can reduce the requirement for the number of qubits. Our numerical results show that our DQLSTM has similar effectiveness to the classical counterpart in a Chinese sentiment analysis task, and outperforms several previous proposals in the same task.**

***Index Terms*—Natural language processing, Quantum machine learning, Quantum long short-term memory neural network**

I. Introduction

Natural language processing (NLP) [1]–[3] is a typical subfield of machine learning (ML) [4]–[6], and applies ML algorithms to deal with linguistic problems such as text and speech processing [7], [8], morphological analysis [9]–[11], sentiment analysis [12]–[14] and etc. As one of the well-known ML models to solve NLP problems, the long short-term memory neural network [15] utilizes a special organization of neural networks to construct a recurrent structure [16], and has the ability to learn the long-range dependencies in sequential data such as texts and voice messages [17], [18]. Such the ability originates from the orderly arrangement of several neural networks whose outputs are combined with multiple nonlinear function. In the meanwhile, one concerned problem in quantum machine learning [19] is how to efficiently generate the nonlinearity of the quantum neural network (QNN) [20], which refers to implement the neural network on a quantum computer. Although, quantum computation has shown its power in the acceleration of classical ML algorithms including support vector machine [21], K-means [22], data-fitting [23], etc. and recent work has demonstrated there exists advantage of QNN compared with the classical neural network [24]. How to design a QNN structure to generate the nonlinearity and further apply it to solve NLP problems still remains an open question.

There have been several successes to utilize quantum algorithms to cope with NLP tasks. For instance, quantum random access memory and quantum nearest-neighbor algorithm have been used as subroutines to design a DisCoCat algorithm, which is a quantum version of the distributional compositional semantics (DisCo) algorithm, in resolving sentence similarity tasks, and have been demonstrated to have a quadratic speed-up [25]. Quantum density matrices have been used to model the dependence between words in a text, and can calculate the correlation between documents [26]. In the meanwhile, a method to build language models based on quantum probability theory has been proposed, and outperforms the classical state-of-the-art method in terms of model complexity and the performance of the automatic speech recognition evaluation tasks [27]. As an important tool for solving ML problems, QNNs have been tried to apply on the NLP tasks. The circuit-centric quantum classifier algorithm can be used to cope with a sentence classification task, which contains more than 100 sentences [28]. A quantum long short-term memory neural network (QLSTM) has successfully translated sentences from English to Persian [29]. However, existing QLSTM models requires the number of qubits is as the same as the dimension of the word vector, the vector to store the sentence information, and uses a simple variational quantum circuit as the substitute of the classical neural network. This method could lead to a surge of the number of qubits when the word vector dimension is high, and has a limited expressive power because of the lack of the nonlinearity generated by the variational quantum circuit.

To address this problem, we utilize the universal duplication-free quantum neural network as a subroutine [30],

Xiaokai Hou and Yingli Yang contributed equally to this work. The authors gratefully acknowledge the grant from National Key R&D Program of China, Grant No. 2018YFA0306703.

978-89-93215-23-6/22/$31.00 ACA

Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:10:29 UTC from IEEE Xplore. Restrictions apply.

and propose a new design of the quantum long short-term memory neural network, a duplication-free quantum long short-term memory neural network (DQLSTM) based on a hybrid quantum classical framework. Specifically, we use the amplitude encoding method to store the classical information, which requires the number of qubits is logarithmic to the dimension of the word vector, and utilizes the classical nonlinear function to keep the expressive power of DQLSTM as the same as the classical LSTM. To demonstrate the effectiveness of our DQLSTM, we numerically implement a sentiment analysis task based on a Chinese E-shopping comments data set. The results demonstrates that the DQLSTM has a better performance than the classical counterpart and outperforms the other QLSTM proposals in terms of the prediction accuracy.

The remainder of the paper is organized as follows. In Section. II, we briefly review the typical NLP problems and the main steps to implement the classical LSTM. In Section. III, we give a detail introduction of the DQLSTM. The implementation of a sentiment analysis task is introduced in Section. IV. Finally, we conclude the paper in Section. V.

II. NLP AND THE CLASSICAL LSTM

Natural language processing (NLP) aims to utilize the machine learning algorithms to help the computers to understand the human language such as texts and voice messages. Typical NLP tasks include translation, text classification, text proofreading, information extraction, speech synthesis, speech recognition, etc. Among them, sentiment analysis is a crucial task, which refers to design the machine learning algorithm to recognize the the sentiment orientation, positive or negative, contained in messages. To cope with such the problem, several methods, e.g., similarity distribution of word clustering [31]–[33], point mutual information (PMI) [34] , naive bayes and suppport vector machine [35], have been proposed.

As the development of the deep learning, neural network has shown its power in dealing with NLP problems. Different from the conventional deep learning tasks such as figure classification and data-fitting, the data sets used in NLP tasks contains the time-related information. It is hard for the naive neural network to process such the information. Hence, the special structure design of neural network is required. The recurrent neural network (RNN) utilizes a recurrent structure of neuron to process the the time-related data, and provides a good ability to make the neural network to remember the relation of each element in a input vector [36]. Based on RNN, there have been several variants, which are widely used in academia and industry, i.e., GRU [37], LSTM [15], BERT [38] and SRU [39]. In this paper, we concentrate on the LSTM. A typical process to implement the classical LSTM in NLP tasks is given below.

A. Preparation of word vector

Given a data set $D = \{(\vec{x}^{(i)}, \vec{y}^{(i)})\}$ where $\vec{x}^{(i)}$ represents a sentence containing m words, $\vec{x}^{(i)} = [x_1^{(i)}, x_2^{(i)}, \cdots, x_m^{(i)}]^T$, and $\vec{y}^{(i)}$ indicates the sentiment label, the first step to implement the LSTM is to prepare a set of word vectors $\{\vec{w}_j^{(i)}\}_{j=1}^m$ with $\vec{w}_j^{(i)} \in R^d$ corresponding to each $\vec{x}^{(i)}$ as the input of the LSTM by using the Skip-Gram method [40].

To explain how to prepare the word vectors, we need to define two sets: i) a corpus set $T = \{x\}$ including all of the words in the data set; ii) a context set $c_j^{(i)} = \{x_{j-2}^{(i)}, x_{j-1}^{(i)}, x_{j+1}^{(i)}, x_{j+2}^{(i)}\}$ containing 4 neighbor elements of a given word $x_j^{(i)}$ in a sentence, $\vec{x}^{(i)}$. The core idea to prepare the word vector is to maximize the conditional probability $p(c_j^{(i)}|x_j^{(i)};\theta)$ by updating the parameters θ where the conditional probability is defined as

$$p(c_j^{(i)}|x_j^{(i)};\theta) = \frac{\sum_{a\in c_j^{(i)}} \exp(\vec{w}_a^T \vec{w}_j^{(i)})}{\sum_{b\in T} \exp(\vec{w}_b^T \vec{w}_j^{(i)})}. \quad (1)$$

Here, $\vec{w}$ represents the word vector. We use a classical neural network whose output layer has d neurons to generate such the word vector. The loss function is the product of Eq. (1) over all of the words $x_j^{(i)}$, and can be written as

$$L(\theta) = \prod_{x_j^{(i)}, c_j^{(i)}} p(c_j^{(i)}|x_j^{(i)};\theta). \quad (2)$$

For convenience, we use the logarithm formula of Eq. (2) in the practice.

B. Implementation of the classical LSTM

Now we have the word vector set $\{\vec{w}_j^{(i)}\}$ and the corresponding label $\vec{y}^{(i)}$ where we use $[0,1]^T$ and $[1,0]^T$ to presented the positive sentiment and the negative sentiment. We put the word vectors into the LSTM, and further train it. As a typical RNN, the LSTM can be implemented as shown in Fig. 1. Given a sentence $\vec{x}^{(i)}$, which consists of m word vectors $[\vec{w}_1^{(i)}, \vec{w}_2^{(i)}, \cdots, \vec{w}_m^{(i)}]$, we execute the unit of LSTM m times to obtain the output. We name the neural network structure as the cell in each execution. In $k-$th time, we take the $\vec{w}_k^{(i)}$, a cell state vector c_k and a hidden state vector h_t as the input. After through the cell, the output are a new cell state vector c_{k+1} and a new hidden state vector h_{k+1}. When we finished m executions, we apply a wight matrix and a sigmoid function on h_{m+1}, and take the value, $\tilde{y}^{(i)} = \sigma(Wh_m)$, as the output of the LSTM. After obtaining the output $\tilde{y}$, we put $\tilde{y}$ into a cross-entropy loss function, and further optimize the parameters in the neural network.

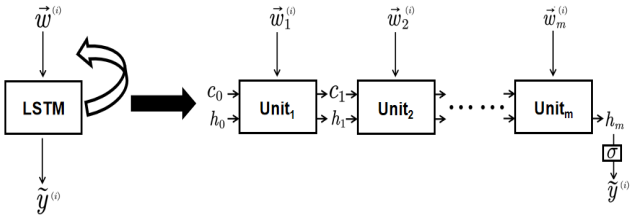


Fig. 1. The implementation of LSTM. LSTM is a typical RNN where the outputs of each unit are taken as the inputs for next unit.

The detail of the cell is shown in Fig. 2. The cell consists of 4 classical neural networks, W_f, W_i, W_c and W_o. After

Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:10:29 UTC from IEEE Xplore. Restrictions apply.

generating the output of each neural network, the cell applies the sigmoid function, σ, and $\tanh$ to the outputs, respectively. After that, the cell uses a series of operations to calculate the new cell state vector c_{k+1} and the new hidden state vector h_{k+1} by using the following rules:

$$\begin{aligned} f_t &= \sigma(W_f \odot v_t \oplus b_f); \\ i_t &= \sigma(W_i \odot v_t + b_i); \\ C_t &= \tanh(W_c \odot v_t \oplus b_c); \\ c_t &= f_t \odot c_{t-1} \oplus i_t \odot C_t; \\ o_t &= \sigma(W_o \odot v_t \oplus b_o); \\ h_t &= o_t \odot \tanh(c_t), \end{aligned}$$

where $\odot$ and $\oplus$ indicate the bitwise multiplication and bitwise addition, respectively.

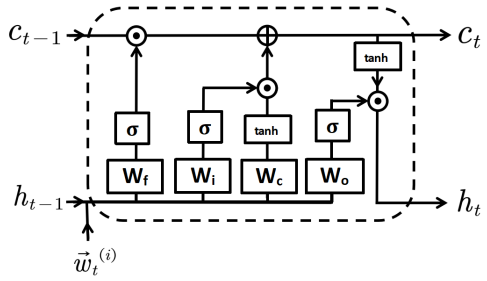


Fig. 2. The detail of cell contained in LSTM.

III. Duplication-free quantum long short-term memory neural network

In this section, we utilize the duplication-free quantum neural network (DQNN) as the substitute of the classical neural network contained in the classical LSTM, and propose a duplication-free quantum long short-term memory neural network (DQLSTM) based on a hybrid classical quantum framework to deal with the NLP tasks. Different from previous quantum long short-term memory neural network (QLSTM) which uses a variational quantum circuit to replace the classical neural network in LSTM, DQNN provides our DQLSTM with the universality, which refers to the ability to approximate any arbitrary function. In the meanwhile, previous QLSTM requires the number of qubits is equal to the dimension of the word vectors. However, in the classical NLP task, the dimension of the word vectors is usually high, which means that encoding the word vectors requires a large number of qubits. To address this problem, we adopt the amplitude encoding method to encode the word vectors. By this way, DQLSTM only requires the number of qubits is logarithmic to the dimension of the word vectors. Hence, DQLSTM can efficiently reduce the number of qubits to deal with the NLP problems. The implementation of DQLSTM is given below.

A. Word vector encoding method

We firstly use the Skip-Gram method (as introduced in Section. II-A) to prepare the word vectors, $\{\vec{w}_j^{(i)}\}$, on a classical computer. For each word vector $\vec{w}_j^{(i)} \in R^d$, we encode it into $\lceil \log_2 d \rceil$ qubits as $|w_j^{(i)}\rangle = \frac{1}{\gamma}\sum_{l=1}^{d} w_{j,l}^{(i)}|l\rangle$ by using the amplitude encoding method [41]. We use $w_{j,l}^{(i)}$ to indicate the $l-$th element in the word vector $\vec{w}_j^{(i)}$. γ is the normalization factor. It is worth to mention that, in order to ensure that the quantum state and the corresponding word vector are one-to-one mapping, we usually add an extra padding information. In this paper, we use the padding data $\tilde{w}_j^{(i)} = \frac{|\vec{w}_j^{(i)}|}{1+|\vec{w}_j^{(i)}|}$. The quantum state can be written as

$$|w_j^{(i)}\rangle = \frac{1}{\gamma}[w_{j,1}^{(i)}, w_{j,1}^{(i)}, \cdots, w_{l,d}^{(i)}, \tilde{w}_j^{(i)}, 0, \cdots, 0]^T \quad (3)$$

with $\gamma = \sqrt{|\vec{w}_j^{(i)}|^2 + |\tilde{w}_j^{(i)}|^2}$.

B. Duplication-free quantum long short-term memory neural network model

Now we have the quantum word vector data set $\{|w_j^{(i)}\rangle\}$ and its corresponding label set $\vec{y}^{(i)}$. The implementation of DQLSTM is similar to the classical counterpart (as shown in Fig. 1). Before executing the DQLSTM, we need to prepare an extra quantum state representing the hidden state h_t. In this paper, we use one qubit to prepare the $|h_t\rangle$, and its initialization is $|h_0\rangle = |+\rangle$. In the meanwhile, we initial the cell state vector c_t as $c_0 = [1, 1]^T$. After the initialization, we use the quantum word vectors, $[|w_1^{(i)}\rangle, |w_2^{(i)}\rangle, \cdots, |w_m^{(i)}\rangle]$, containing in the sentence $\vec{x}^{(i)}$ as the input and execute the cell m times to generate the output.

As for k-th unit, its inputs are the tensor product of the quantum word vector state $|w_k^{(i)}\rangle$ and the quantum hidden state $|h_{k-1}\rangle$. By feeding $|w_k^{(i)}\rangle \otimes |h_{k-1}\rangle$ into the cell, we can obtain the new cell state vector c_k and the new quantum hidden state $|h_k\rangle$. After m executions, we measure the final quantum hidden state $|h_m\rangle$ by using $\Pi_0 = |0\rangle\langle 0|$ and $\Pi_1 = |1\rangle\langle 1|$, and obtain the measurement result $p^{(i)} = [p_0^{(i)}, p_1^{(i)}]^T$. Then, we apply a linear transformation and a sigmoid function to generate the output as $\tilde{y}^{(i)} = \sigma(Wp^{(i)})$. As an estimator of the label $y^{(i)}$, we put the $\tilde{y}^{(i)}$ and $y^{(i)}$ into a loss function:

$$L(\vec{\theta}, \vec{a}, \vec{c}, W) = \sum_{i}^{n_{batch}} l(\tilde{y}^{(i)}, y^{(i)}) \quad (4)$$

where $\vec{\theta}, \vec{a}, \vec{c}, W$ are the trainable parameters and will be introduced in the following. l is the cross-entropy between $\tilde{y}^{(i)}$ and $y^{(i)}$. n_{batch} indicates the batch number of sentences which we need to use in one training iteration. After calculating the loss function, we can minimize the loss function by using the optimization methods such as ADAM [42], NGD [43] and SPSA [44], etc. In this work, we adopt the ADAM algorithm to train our DQLSTM.

The cell contained in DQLSTM is given in given in Fig. 3. In the cell, we use 4 duplication-free quantum neural networks (DQNN) as the substitutes contained in the classical LSTM. As for each duplication-free quantum neural network,

Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:10:29 UTC from IEEE Xplore. Restrictions apply.

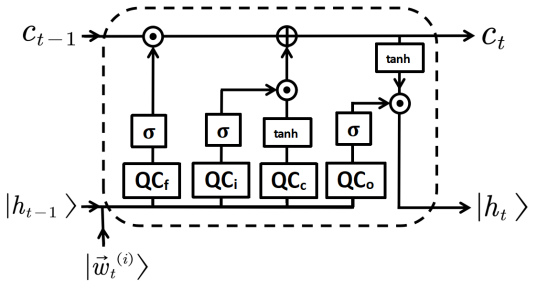


Fig. 3. The detail of cell contained in DQLSTM. QC indicates a variational quantum classifier with multiple measurement operations by using several observables.

it consists a QC, which contains a variational quantum circuit and multiple measurement operations based on several measurement observables, and some parameterized classical nonlinear nodes whose number is equal to the number of the measurement observables. The implementation of the DQNN can be described in the following way. Firstly, we use the state $|w_k^{(i)}\rangle \otimes |h_{k-1}\rangle$ as the input, and apply a variational quantum circuit $U(\theta)$ on it. The circuit ansatz is given in Fig. 4 where G is a general parameterized single-qubit rotation gate and CG is a parameterized general control rotation gate, $CG(\theta_1, \theta_2, \theta_3) = |0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes G(\theta_1, \theta_2, \theta_3)$, with

$$G(\theta_1, \theta_2, \theta_3) = \begin{pmatrix} e^{i\theta_2} cos(\theta_1) & e^{i\theta_3} sin(\theta_1) \\ -e^{i\theta_3} sin(\theta_1) & e^{-i\theta_2} cos(\theta_1) \end{pmatrix}.$$

After applying such the circuit, we measure the average energy $\langle B_i \rangle$ based a set of measurement observables $\{B_i\}^N$ according to $\langle B_i \rangle = Tr[U(\theta)|w_k^{(i)}\rangle \otimes |h_{k-1}\rangle\langle h_{k_1}| \otimes \langle w_k^{(i)}|U(\theta)^\dagger B_i]$,where each B_i is randomly sampled from the complete Pauli basis. The implementation of QC is given in Fig. 5. Trough a QC, we obtain a series of expectation values $\langle B_j \rangle_{j=1}^N$, and further feed these values into a nonlinear function. The nonlinear function used in DQLSTM has two formula: i) $\tanh$; ii) a parameterized sigmoid function,

$$\sigma(\langle B_i \rangle) = \frac{1}{1 + exp\{-a(\langle B_i \rangle - c)\}}$$

where a and c are the trainable parameters, and satisfy $a > 2$, $1 > c > 0$.

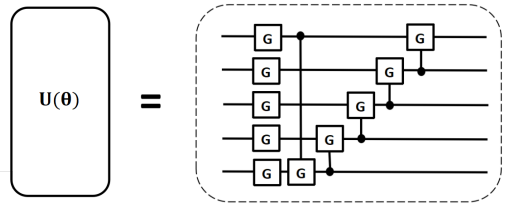


Fig. 4. The structure of parameterized quantum circuit used in DQNN.

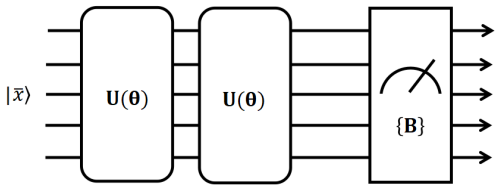


Fig. 5. The implementation of each QC. We take a 5-qubit circuit with depth 2 as an example.

Now we have executed the 4 DQNNs, and need to calculate the outputs of each cell according to the following rules:

$$f_t = \sigma(\langle B_f \rangle) \tag{5}$$
$$i_t = \sigma(\langle B_i \rangle) \tag{6}$$
$$\tilde{C}_t = \tanh(\langle B_c \rangle) \tag{7}$$
$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{C} \tag{8}$$
$$o_t = \sigma(\langle B_o \rangle) \tag{9}$$
$$h_t = o_t \otimes \tanh(c_t) \tag{10}$$
$$\tilde{y} = \sigma(W \otimes h_m) \tag{11}$$

where we use $\langle B_j \rangle$, $j \in f, i, c, O$ indicates the measurement results from different DQNNs, and $\odot$, $\oplus$ represent the bitwise multiplication and bitwise addition, respectively. It is worth to mention that, when we obtain the new h_t, we need to encode it into a new quantum hidden state $|h_t\rangle$ by utilizing a parameterized general single-qubit rotation.

C. Explanations for 4 DQNNs

Similar with the classical LSTM, the 4 DQNNs in the DQLSTM have different functions to cope with the time-related data. In this section, we will introduce each DQNN and its function below:

i) QC_f (Forget gate): QC_f is the forget gate, and is applied on the input states, $|w_k^{(i)}\rangle \otimes |h_{k-1}\rangle$. The output of QC_f is shown in Eq. (5). The value, f_t, determines how much information should be "forgotten" or "retained" in c_{t-1} according to $f_t \odot c_{t-1}$. Specially, when $f_t = 1$, the c_{t-1} remains unchanged, and all of the information in c_{t-1} can be retained; in the meanwhile, when $f_t = 0$, the c_{t-1} will be set to 0, and all of the information in c_{t-1} will be forgotten. The range of f_t is $[0, 1]$, which means that partial information in c_{t-1} will be retained in t-th unit.

ii) QC_i, QC_c (Input gate and update gate): QC_i and QC_c represent the input gate and the update gate, respectively. The combination of the their outputs can determine how much information in the $t-$th unit should be added to the cell state vector c_t. The output of QC_i is given in Eq. (6) and the output of QC_c is given in Eq. (7). According to Eq. (8), we combine their outputs with the forgot gate and c_{t-1}, and further generate a new cell state vector, c_t, which is passed to the next unit.

iii) QC_o (Output gate): The aim of QC_o is to generate the output of the cell. After obtaining the output of QC_o (shown

Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:10:29 UTC from IEEE Xplore. Restrictions apply.

in Eq. (9)), o_t, we use the updated cell state vector, c_t, which is gotten by the input gate and update gate, and the o_t to generate the new hidden state vector, h_t, for the next unit based on Eq. (10). When we execute the $k-$th unit, $k \neq m$, we normalize h_t, and prepare the corresponding quantum hidden state $|h_t\rangle$ for the next cell. When we execute the final unit, we apply a linear transformation, W, and further use a sigmoid function to generate the output of the DQLSTM, i.e. $\tilde{y}$.

IV. Numerical simulations

In this section, we numerically compare the performance of our DQLSTM with the classical LSTM model [15] over a Chinese E-commerce comments data set, and further compare our model with two kinds of QLSTM models with the same number of layers as our DQLSTM in the same task . One of the QLSTM models is proposed in ref. [45], and we name it as the QLSTM in the following. The other utilize the circuit-centric quantum classifier [46] as the subroutines, QC_f, QC_i, QC_c,QC_o, in the DQLSTM, and we name it as CQLSTM below. The variational circuit used in these three models consists of 2 layers of the circuit as shown in Fig. (4).

The data set contains $10,000$ comments, which are obtained from Taobao, with positive and negative sentiment labels. The distribution of the comment length and the number of each sentiment category is shown Fig. IV Each sentiment category consists of $5,000$ comments. Before implementing the numerical simulations, we segment each comments into several words by using the Jieba library, and further utilize the Skip-Gram method to prepare the word vectors. We divide the whole data set into two parts as the training set and the test set. The training set consists of $9,000$ comments, in the meanwhile, the test set has $1,000$ comments. In both of them, the ratio of positive and negative comments is $1:1$.

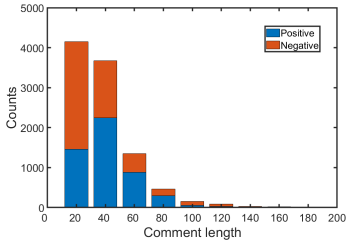


Fig. 6. The distribution of the comment length and the number of each sentiment category.The horizontal axis represents the length of each comment, and the vertical axis represents the number of comments.

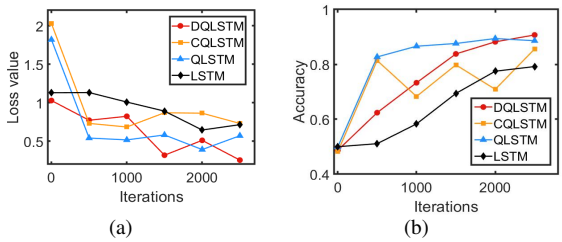
To compare the performance of three quantum models, we encode a word vector into 4 qubits. For DQLSTM and CQLSTM, both of them use the amplitude-encoding method to store the classical information. Hence we generate a 15-dimensional word vector $\vec{w}_j^{(i)}$ for each comment $x^{(i)}$ and prepare the input state $|w_j^{(i)}\rangle$ by using Eq. (3). In addition, we use one extra qubit to store the hidden state vector h. Since the difference of the nonlinearity generation methods between DQLSTM and CQLSTM, i.e. the DQLSTM needs to use the classical sigmoid function and the CQLSTM needs to duplicate the same input state into multiple subsystems, we use one more copy in the CQLSTM. Hence for each component in the LSTM framework, QC_f, QC_i, QC_c and QC_o, DQLSTM uses a 5-qubit state $|w_j^{(i)}\rangle \otimes |h\rangle$ as its input and CQLSTM uses a 9-qubit state $|w_j^{(i)}\rangle^{\otimes 2} \otimes |h\rangle$ as its input. For QLSTM, we generate a 4-dimensional word vector $\vec{w}_j^{(i)}$ corresponding to each comment $x^{(i)}$ and prepare the input state $|w_j^{(i)}\rangle$ by using

$$|w_j^{(i)}\rangle = \otimes_{d=1}^{4} R_z(\arctan(w_{j,d}^{(i)2}))R_y(\arctan(w_{j,d}^{(i)}))H|0\rangle \tag{12}$$

Two extra qubits are required to store the hidden state vector h. Hence the input of each component in QLSTM is a 6-qubit state $|w_j^{(i)}\rangle \otimes |h\rangle$. The specific configuration of each model is shown in Table. I. In addition, we further compare our DQLSTM with the classical LSTM with 15-dimensional word vector as its input and 264 parameters. The comparison is given below.

TABLE I
The specific configurations of each model

Model	Word vector dim	# Total Qubits	# Layers	# Gates
DQLSTM	15	20	2	240
CQLSTM	15	36	2	432
QLSTM	4	24	2	288



(a) (b)

Fig. 7. The numerical simulations of different models. (a) The relation between the loss function and the number of iterations on the train set. (b) The relation between the loss function and the number of iterations on the test set.

In the numerical simulations, we use the ADAM algorithm to optimize the loss function with the batch size, 32, learning rate, 0.01. The classical LSTM model has 2 hidden layers. Each layer consists of 30 neurons. It can be seen from Fig. IV that, after 2500 iterations, the DQLSTM achieves a lower loss value, 0.5473, than 0.61 (QLSTM), 0.6447 (LSTM) and 0.7536 (CQLSTM) on the training set among all of the models. In the meanwhile, the DQLSTM has a higher prediction accuracy, 90.7%, than 88.6% (QLSTM), 79.2% (LSTM) and 85.6% (CQLSTM). The results shows that the convergence rate of DQLSTM is the higher than the classical LSTM model, but is lower than QLSTM and CQLSTM model. To summarize, our DQLSTM outperforms the other models in terms of the loss value and the accuracy, but requires an extra iterations to converge. In the meanwhile, from Table I, we can find that our DQLSTM uses fewer qubits and quantum gates compared with the QLSTM and the CQLSTM. This advantage

Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:10:29 UTC from IEEE Xplore. Restrictions apply.

originates from the different nonlinearity generation method used in the DQLSTM. Utilizing the classical sigmoid function leads our DQLSTM can find the pattern hidden in the data set more easier, and requires fewer quantum resources.

V. CONCLUSION

In this paper, we propose a new design of quantum long-short-term memory neural network model based the universal duplication free quantum neural network. The universality provided by the duplication free quantum neural network makes our DQLSTM has the same ability as the classical LSTM model. In the meanwhile, previous QLSTM proposal requires the number of qubits is equal to the dimension of the classical word vector. To address this problem, we utilize the amplitude encoding method to store the classical word vector, and decrease the requirement of the number of qubits. We numerically compare the performance of the DQLSTM, the classical LSTM and other QLSTM models. The results demonstrate that our proposal has the same ability as the classical LSTM model and has a better performance than the other QLSTM models over a Chinese sentiment analysis task. We expect our DQLSTM can be used in resolving more NLP problems.

REFERENCES

[1] Elizabeth D Liddy. Natural language processing. 2001.

[2] Gobinda G Chowdhury. Natural language processing. *Annual review of information science and technology*, 37(1):51–89, 2003.

[3] Prakash M Nadkarni, Lucila Ohno-Machado, and Wendy W Chapman. Natural language processing: an introduction. *Journal of the American Medical Informatics Association*, 18(5):544–551, 2011.

[4] Michael I Jordan and Tom M Mitchell. Machine learning: Trends, perspectives, and prospects. *Science*, 349(6245):255–260, 2015.

[5] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. Machine learning basics. *Deep learning*, 1(7):98–164, 2016.

[6] David E Goldberg and John Henry Holland. Genetic algorithms and machine learning. 1988.

[7] Rolf Carlson, B Granstrom, and Sheri Hunnicutt. A multi-language text-to-speech module. In *ICASSP'82. IEEE International Conference on Acoustics, Speech, and Signal Processing*, volume 7, pages 1604–1607. IEEE, 1982.

[8] Sadaoki Furui, Tomonori Kikuchi, Yosuke Shinnaka, and Chiori Hori. Speech-to-text and speech-to-speech summarization of spontaneous speech. *IEEE Transactions on Speech and Audio Processing*, 12(4):401–408, 2004.

[9] Jeremy M Anglin, George A Miller, and Pamela C Wakefield. Vocabulary development: A morphological analysis. *Monographs of the society for research in child development*, pages i–186, 1993.

[10] Tom Ritchey. General morphological analysis (gma). In *Wicked problems–Social messes*, pages 7–18. Springer, 2011.

[11] Taku Kudo, Kaoru Yamamoto, and Yuji Matsumoto. Applying conditional random fields to japanese morphological analysis. In *Proceedings of the 2004 conference on empirical methods in natural language processing*, pages 230–237, 2004.

[12] Bing Liu et al. Sentiment analysis and subjectivity. *Handbook of natural language processing*, 2(2010):627–666, 2010.

[13] Ronen Feldman. Techniques and applications for sentiment analysis. *Communications of the ACM*, 56(4):82–89, 2013.

[14] Rudy Prabowo and Mike Thelwall. Sentiment analysis: A combined approach. *Journal of Informetrics*, 3(2):143–157, 2009.

[15] Jürgen Schmidhuber, Sepp Hochreiter, et al. Long short-term memory. *Neural Comput*, 9(8):1735–1780, 1997.

[16] Jeffrey L Elman. Finding structure in time. *Cognitive science*, 14(2):179–211, 1990.

[17] Alex Graves and Jürgen Schmidhuber. Framewise phoneme classification with bidirectional lstm and other neural network architectures. *Neural networks*, 18(5-6):602–610, 2005.

[18] Zhiheng Huang, Wei Xu, and Kai Yu. Bidirectional lstm-crf models for sequence tagging. *arXiv preprint arXiv:1508.01991*, 2015.

[19] Jacob Biamonte, Peter Wittek, Nicola Pancotti, Patrick Rebentrost, Nathan Wiebe, and Seth Lloyd. Quantum machine learning. *Nature*, 549(7671):195–202, 2017.

[20] Subhash C Kak. Quantum neural computing. *Advances in imaging and electron physics*, 94:259–313, 1995.

[21] Patrick Rebentrost, Masoud Mohseni, and Seth Lloyd. Quantum support vector machine for big data classification. *Physical review letters*, 113(13):130503, 2014.

[22] Esma Aïmeur, Gilles Brassard, and Sébastien Gambs. Machine learning in a quantum world. In *Conference of the Canadian Society for Computational Studies of Intelligence*, pages 431–442. Springer, 2006.

[23] Nathan Wiebe, Daniel Braun, and Seth Lloyd. Quantum algorithm for data fitting. *Physical review letters*, 109(5):050505, 2012.

[24] Hsin-Yuan Huang, Michael Broughton, Masoud Mohseni, Ryan Babbush, Sergio Boixo, Hartmut Neven, and Jarrod R McClean. Power of data in quantum machine learning. *Nature communications*, 12(1):1–9, 2021.

[25] William Zeng and Bob Coecke. Quantum algorithms for compositional natural language processing. *arXiv preprint arXiv:1608.01406*, 2016.

[26] Alessandro Sordoni, Jian-Yun Nie, and Yoshua Bengio. Modeling term dependencies with quantum language models for ir. In *Proceedings of the 36th international ACM SIGIR conference on Research and development in information retrieval*, pages 653–662, 2013.

[27] Ivano Basile and Fabio Tamburini. Towards quantum language models. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 1840–1849, 2017.

[28] Robin Lorenz, Anna Pearson, Konstantinos Meichanetzidis, Dimitri Kartsaklis, and Bob Coecke. Qnlp in practice: Running compositional models of meaning on a quantum computer. *arXiv preprint arXiv:2102.12846*, 2021.

[29] Mina Abbaszade, Vahid Salari, Seyed Shahin Mousavi, Mariam Zomorodi, and Xujuan Zhou. Application of quantum natural language processing for language translation. *IEEE Access*, 9:130434–130448, 2021.

[30] Xiaokai Hou, Guanyu Zhou, Qingyu Li, Shan Jin, and Xiaoting Wang. A universal duplication-free quantum neural network. *arXiv preprint arXiv:2106.13211*, 2021.

[31] Vasileios Hatzivassiloglou and Janyce Wiebe. Effects of adjective orientation and gradability on sentence subjectivity. In *COLING 2000 Volume 1: The 18th International Conference on Computational Linguistics*, 2000.

[32] Janyce Wiebe, Rebecca Bruce, Matthew Bell, Melanie Martin, and Theresa Wilson. A corpus study of evaluative and speculative language. In *Proceedings of the Second SIGdial Workshop on Discourse and Dialogue*, 2001.

[33] Janyce Wiebe et al. Learning subjective adjectives from corpora. *Aaai/iaai*, 20(0):0, 2000.

[34] Peter D Turney and Michael L Littman. Measuring praise and criticism: Inference of semantic orientation from association. *acm Transactions on Information Systems (tois)*, 21(4):315–346, 2003.

[35] Bo Pang, Lillian Lee, and Shivakumar Vaithyanathan. Thumbs up? sentiment classification using machine learning techniques. *arXiv preprint cs/0205070*, 2002.

[36] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *nature*, 323(6088):533–536, 1986.

[37] Kyunghyun Cho, Bart Van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*, 2014.

[38] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.

[39] Tao Lei, Yu Zhang, and Yoav Artzi. Training rnns as fast as cnns. 2018.

[40] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.

[41] Takahiro Goto, Quoc Hoan Tran, and Kohei Nakajima. Universal approximation property of quantum feature map. *arXiv preprint arXiv:2009.00298*, 2020.

[42] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:10:29 UTC from IEEE Xplore. Restrictions apply.

[43] Razvan Pascanu and Yoshua Bengio. Revisiting natural gradient for deep networks. *arXiv preprint arXiv:1301.3584*, 2013.
[44] James C Spall et al. Multivariate stochastic approximation using a simultaneous perturbation gradient approximation. *IEEE transactions on automatic control*, 37(3):332–341, 1992.
[45] Samuel Yen-Chi Chen, Shinjae Yoo, and Yao-Lung L Fang. Quantum long short-term memory. *arXiv preprint arXiv:2009.01783*, 2020.
[46] Maria Schuld, Alex Bocharov, Krysta M Svore, and Nathan Wiebe. Circuit-centric quantum classifiers. *Physical Review A*, 101(3):032308, 2020.

Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:10:29 UTC from IEEE Xplore. Restrictions apply.